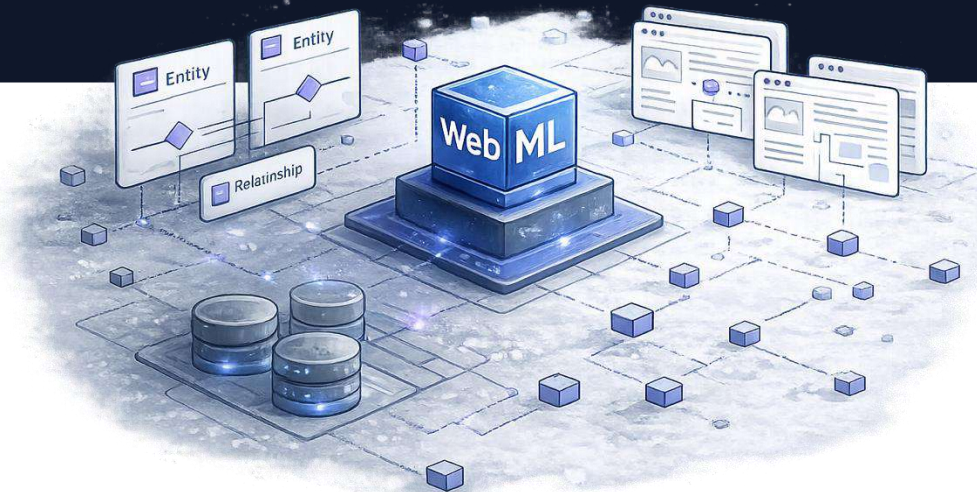




Progettazione di applicazioni web full stack

Progettazione model-driven dei contenuti informativi



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia



Argomento di questa lezione...

Cosa faremo

- Introduzione della **metodologia WebML** per la progettazione di applicazioni web data-intensive

Output

- Modello dei dati progettato in accordo con i principi della metodologia
- Codifica della logica di backend guidata dalla metodologia utilizzando **NestJS** e **TypeORM**



- Metodologia di sviluppo per applicazioni web *data-intensive*
- WebML (*Web Modeling Language*)
 - Modellazione del contenuto informativo
 - Modellazione dell'ipertesto
 - Guida la codifica della logica di backend e della logica di frontend
 - Passo avanti nella proposta di un linguaggio per la modellazione congiunta di dati e ipertesto rispetto alla Guida di Yale



- Riferimento classico per sviluppare siti web chiari, efficaci e usabili
 - Strategia e pianificazione (obiettivi, utenti target, organizzazione dei contenuti)
 - Progettazione dell'interfaccia e design visivo
 - Progettazione e implementazione dei contenuti
 - Analisi di usabilità (facilità d'uso e orientamento dell'utente)
 - Accessibilità
 - Gestione e manutenzione

«Web Style Guide: Basic Design Principles for Creating Web Sites» - P.J. Lynch and S. Horton, Yale University Press, 1999



- La metodologia prevede i seguenti passaggi:
 - Specifica dei requisiti
 - Gruppi utente e casi d'uso (funzionalità)
 - Dizionario dei dati
 - Progettazione dei dati
 - Progettazione dell'ipertesto
 - Implementazione
 - Promozione, monitoraggio, valutazione, manutenzione



- **IN**: requisiti di business
 - **OUT**: specifiche semi-formali, con un buon livello di precisione, ma facili da comprendere anche dal cliente
-
- Due sotto-attività principali:
 - Raccolta dei requisiti
 - Analisi dei requisiti



- Attività poco strutturata, mirata a collezionare le seguenti informazioni:
 - Utenti e gruppi utenti (e.g., interni, esterni)
 - Requisiti funzionali (casi d'uso)
 - Requisiti sui dati (dizionario)
 - Mockup dell'interfaccia web
- Requisiti non funzionali:
 - Usabilità, prestazioni, disponibilità, scalabilità, sicurezza, manutenibilità, accesso da parte di più dispositivi differenti

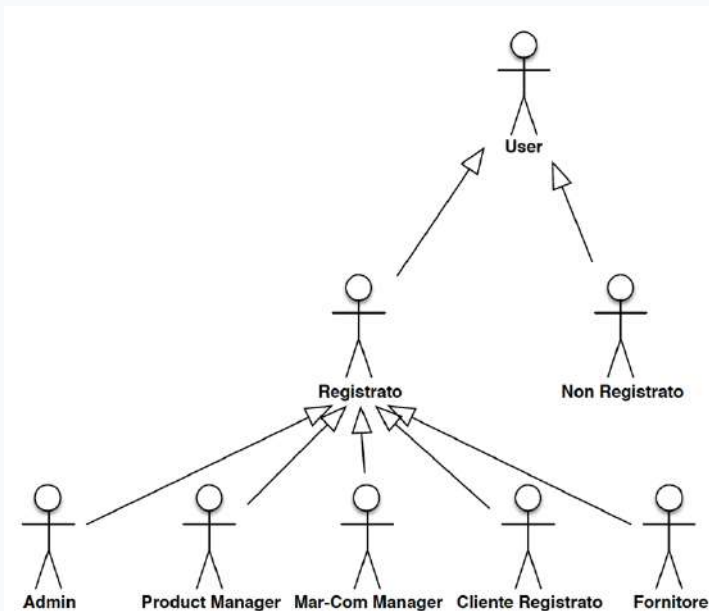


- Revisione e formalizzazione dei requisiti raccolti e produzione di un insieme di specifiche semi-formali tramite l'utilizzo di opportuni linguaggi, convenzioni, formalismi
 - UML per gruppi di utenti e casi d'uso
 - Soluzioni ad-hoc per il dizionario dei dati e i mockup dell'interfaccia Web

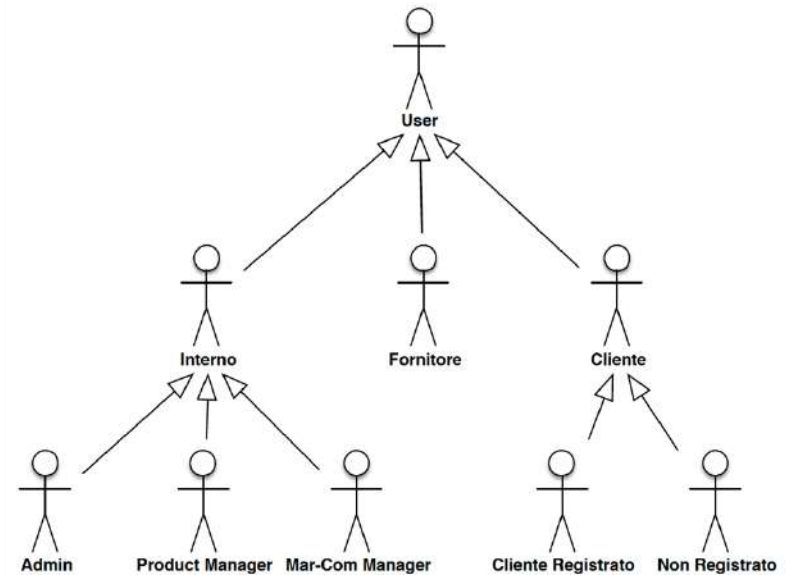
Gerarchia dei gruppi utente



- Focus su registrazione



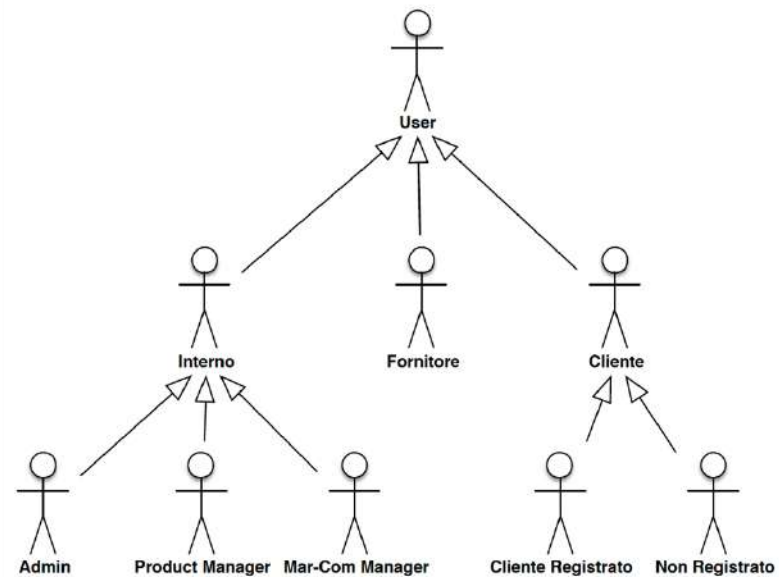
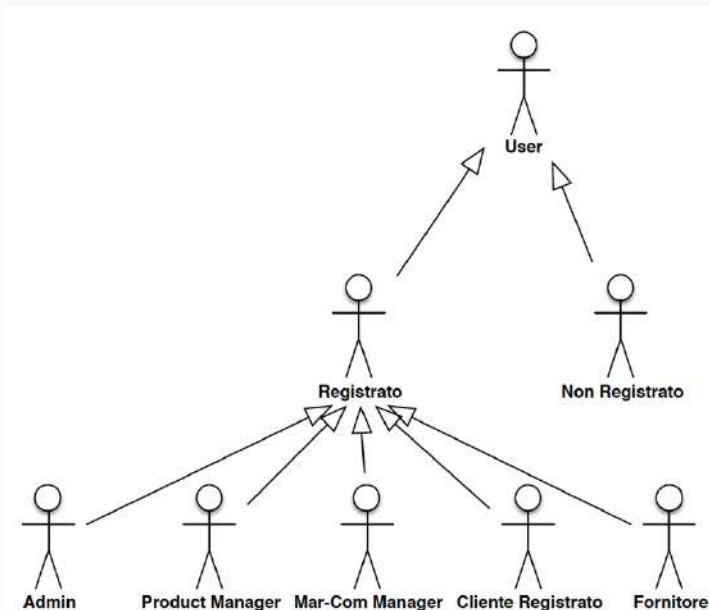
- Internet vs Intranet vs Extranet



Gerarchia dei gruppi utente



Va sempre identificato almeno un gruppo target a cui è dedicata l'applicazione (utile per focalizzarne meglio le funzionalità)



Specifica dei gruppi utente e casi d'uso



Nome del gruppo	MarCom Manager
Descrizione:	Marketing & Communication, inserimento di nuovo personale, gestione delle news
Dati di profilo:	Nome, cognome, email, indirizzo ufficio
Super-gruppo:	Utente registrato
Sotto-gruppo:	Nessuno (nodo foglia nella gerarchia degli utenti)
Oggetti in lettura:	Product, News, CategoriaNews
Oggetti in scrittura:	News, CategoriaNews
Casi d'uso rilevanti:	«CRUD News», «CRUD CategoriaNews», «Modifica dati profilo»

- È auspicabile sfruttare il *meccanismo di ereditarietà* tra i gruppi utente



- Domande tipiche:
 1. Quali sono gli oggetti informativi da pubblicare/gestire tramite l'applicazione?
 2. Quali proprietà li caratterizzano?
 3. In che modo i vari oggetti sono collegati?



- Elenco degli *elementi informativi principali*, individuati durante la raccolta dei requisiti sui dati
- Per ogni elemento nel **dizionario**, è auspicabile specificare:
 - Nome ed eventuali sinonimi
 - Descrizione con istanze di esempio
 - Proprietà (attributi)
 - Relazioni con altri elementi (incluse le relazioni di «*generalizzazione/specializzazione*»)
 - Componenti (esplicitazione del concetto di «*part-of*»)



- La definizione di prototipi di interfaccia («*freddi*» o «*tiepidi*») supporta la raccolta e analisi dei requisiti
 - **Prototipo «freddo» (low-fidelity)** – wireframe realizzati con strumenti dedicati (es. Balsamiq), rappresentazione semplificata della struttura delle pagine, focus su layout, contenuti e navigazione (non sul design grafico)
 - **Prototipo «tiepido» (mid-fidelity)** - pagine web statiche (HTML/CSS) con eventuali interazioni di base (Javascript), maggiore aderenza al risultato finale, utile per validare comportamento, struttura e prime scelte visive



- L'**analisi e specifica** dei requisiti è utile per definire le prime «*rotte dell'applicazione*» (CRUD):
 - Oggetti in lettura → azioni READ
 - Oggetti in scrittura + casi d'uso → azioni CREATE-UPDATE-DELETE
 - Altri casi d'uso → rotte ulteriori che verranno raffinate nei passi successivi
- La **mappatura** tra gruppi utente e casi d'uso permette di definire le opportune «*guardie*» sulle rotte
 - La mappatura consente anche di individuare possibili errori di progettazione (ci sono oggetti che nessuno legge? Ci sono oggetti che nessuno crea/aggiorna?)

Esempio – Biblioteca online (I)



	Utente visitatore	Utente registrato	Utente bibliotecario	Admin
Book	R	(CUD)R	CRUD	
Author	R	(CUD)R	CRUD	
Address	R	(CUD)R	CRUD	
Category	R	R	CRUD	
Utente registrato		(CRUD)	CRUD	CRUD
Utente bibliotecario			(CRUD)	CRUD

Esempio – Biblioteca online (I)



- L'utente registrato può visualizzare R tutti i libri, ma li può modificare (creare, aggiornare, cancellare) solo con limitazioni (es. solo quelli inseriti da lui, se lo prevede l'applicazione)

		Utente registrato	Utente bibliotecario	Admin
		(CUD)R	CRUD	
Author	R	(CUD)R	CRUD	
Address	R	(CUD)R	CRUD	
Category	R	R	CRUD	
Utente registrato			CRUD	CRUD
Utente bibliotecario			(CRUD)	CRUD

- Il profilo dell'utente bibliotecario può essere modificato solo dal bibliotecario stesso (ognuno gestisce solo il proprio profilo) oppure dall'amministratore del sito (tutti i profili)

Esempio – Biblioteca online (con errori)



	Utente visitatore	Utente registrato	Utente bibliotecario	Admin
Book	R	(CUD)R	CRUD	
Author	R	(CUD)R	CRUD	
Address	R	R	R	
Utente registrato		R	CRUD	
Utente bibliotecario		(CRUD)	CRUD	CRUD
			(CRUD)	CRUD

- Chi gestisce gli indirizzi degli autori? Molti ruoli li possono leggere, nessuno li può creare, cancellare, modificare

Esempio – Biblioteca online (con errori?)



	Utente visitatore	Utente registrato	Utente bibliotecario	Admin
Book		(CUD)R	CRUD	
Author		(CUD)R	CRUD	
Address		R	R	
Category				
Utente registrato				
Utente bibliotecario				

- Ma serve veramente un utente amministratore? Nessun oggetto in lettura e scrittura
- In applicazioni non unicamente data-intensive la decisione di toglierlo viene demandata agli step successivi, ma qui scatta un **ALERT**

Esempio – Biblioteca online (con errori?)



	Utente visitatore	Utente registrato	Utente bibliotecario	Admin
Book		(CUD)R	CRUD	
Author		(CUD)R	CRUD	
Address		R	R	
Category				
Utente registrato				
Utente bibliotecario			(CRUD)	

Un progetto in cui l'utente visitatore non vede e non modifica nulla è tipico dei sito «posizionati» completamente dietro il meccanismo di autenticazione



- La modellazione concettuale dei dati in WebML si appoggia su primitive del **modello ER**:
 - **Entità**: classi di oggetti omogenei nel dominio applicativo
 - **Attributi**: proprietà delle entità (con tipo semplice)
 - **Relazioni**: connessioni tra entità (con cardinalità)
 - **Gerarchia IS-A**: costruito usato per classificare o raggruppare

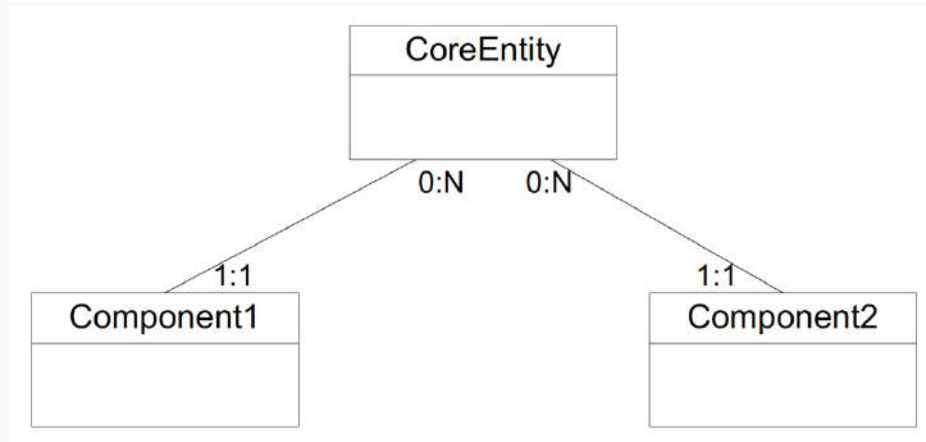
Progettazione dei dati (II)



- **IN**: dizionario dei dati, mockup dell'interfaccia web, requisiti funzionali e requisiti utente
 - **OUT**: schema formale dei dati (schema ER/UML)
-
- Può trarre vantaggio da una classificazione degli oggetti informativi da rappresentare nello schema in base al loro utilizzo:
 - **Core**: i contenuti informativi principali del sito
 - **Interconnessione**: per passare da un contenuto informativo all'altro
 - **Accesso**: per permettere all'utente di raggiungere il più velocemente possibile i contenuti informativi core
 - **Personalizzazione**: per adattare parti diverse del sito web alle esigenze di attori diversi



- Oggetti informativi principali gestiti dall'applicazione (dal punto di vista dei *gruppi utente target*):
 - *Entità centrali* attorno alle quali si sviluppa l'intero schema
 - *Più di una singola entità* – rappresentazione di proprietà complesse e di componenti





- **Sottoschema «core»**: insieme di entità connesse da relazioni, che collettivamente rappresentano un singolo concetto «core»
- Partendo da un concetto «core» nel dizionario dei dati:
 1. Il concetto «core» è rappresentato da un'entità, detta **entità «core»**
 2. Le proprietà con valore singolo e atomico diventano **attributi** dell'entità «core»; le proprietà chiave diventano **chiavi** dell'entità «core»
 3. Le proprietà con valori multipli o strutturati diventano **componenti** interni dell'entità core



- I componenti interni sono rappresentati come entità associate all'entità core attraverso una relazione
- I vincoli di cardinalità delle relazioni che associano il componente all'entità core possono creare due casi:
 1. relazione con vincolo di cardinalità 1:1 dalla parte del componente; il componente è una sotto-parte propria del concetto core, *nessuna istanza del componente interno può esistere in assenza dell'entità core a cui è associata*; inoltre, la stessa istanza di un componente interno non può essere condivisa da più oggetti core; i componenti interni sono chiamati *entità deboli* oppure *componenti part-of*

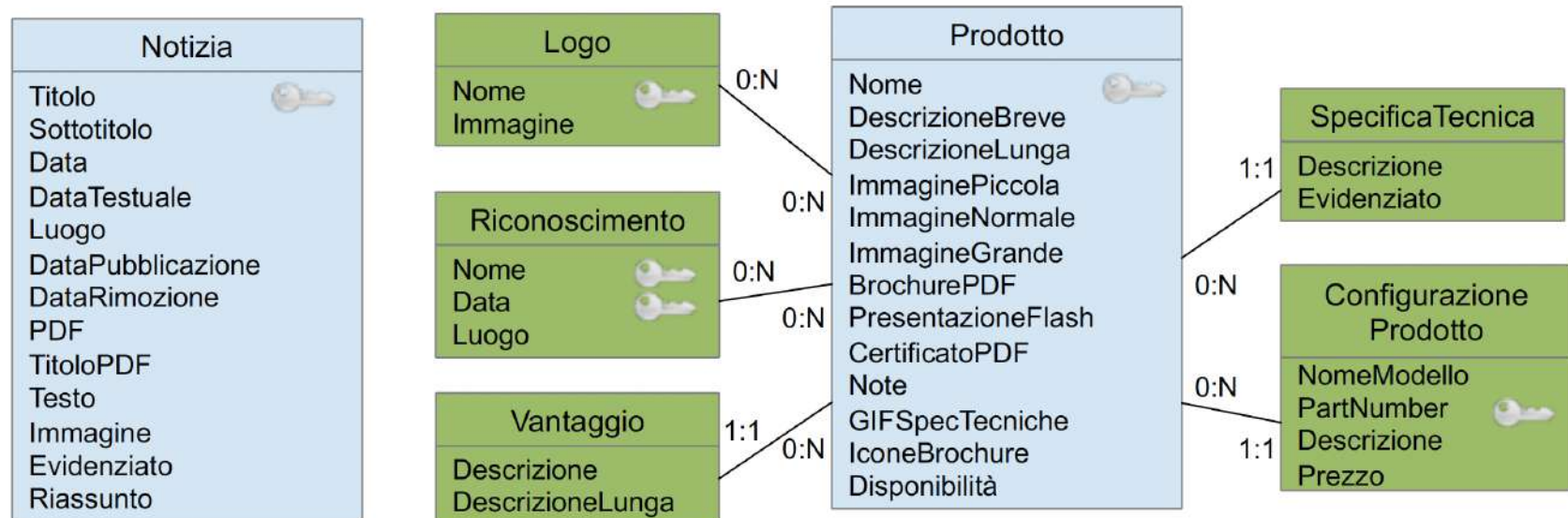


2. relazione con vincolo di cardinalità 0:N dalla parte del componente; il componente è considerato una parte del concetto core, *una sua istanza può esistere indipendentemente dall'associazione ad una istanza dell'entità core, oppure può essere condivisa tra entità core diverse*; il componente interno non può tuttavia essere considerato un elemento centrale e a sé stante e quindi non può essere elevato al ruolo di entità core

Sottoschema «core» (IV)



- Progetto dettagliato del sottoschema «core»: attributi, componenti e relazioni



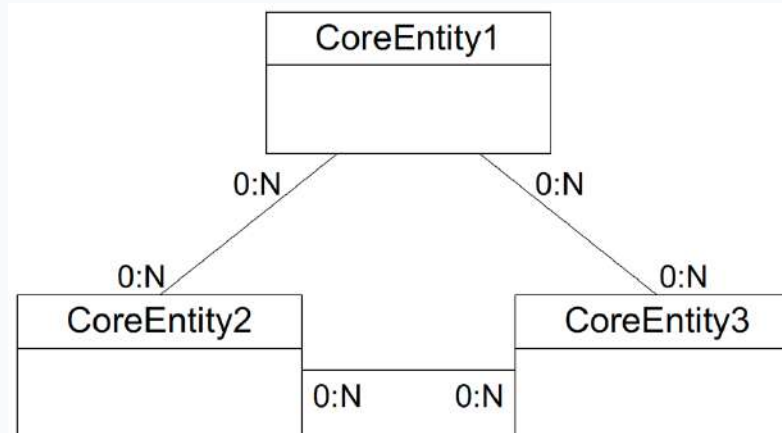


- L'individuazione di un sottoschema core suggerisce delle linee guida di buona progettazione, con un impatto anche sulla parte ipertestuale:
 - Deve essere prevista una pagina di dettaglio per ogni sottoschema core e corrispondente utente target
 - Va aggiunta una rotta che legge l'entità core e i suoi componenti
 - Può essere aggiunta una pagina di dettaglio del componente (e rotta corrispondente), ma tale pagina deve sempre prevedere anche le informazioni sulla corrispondente entità core
 - Nel caso di entità deboli, la loro creazione avviene sempre contestualmente alla creazione dell'entità core (fusione delle rotte di creazione), la cancellazione dell'entità core comporta sempre una cancellazione a cascata

Oggetti di «interconnessione»



- Associazioni semantiche tra entità «core», usati per definire link e indici per la navigazione tra entità «core»



Sottoschema di «interconnessione» (I)

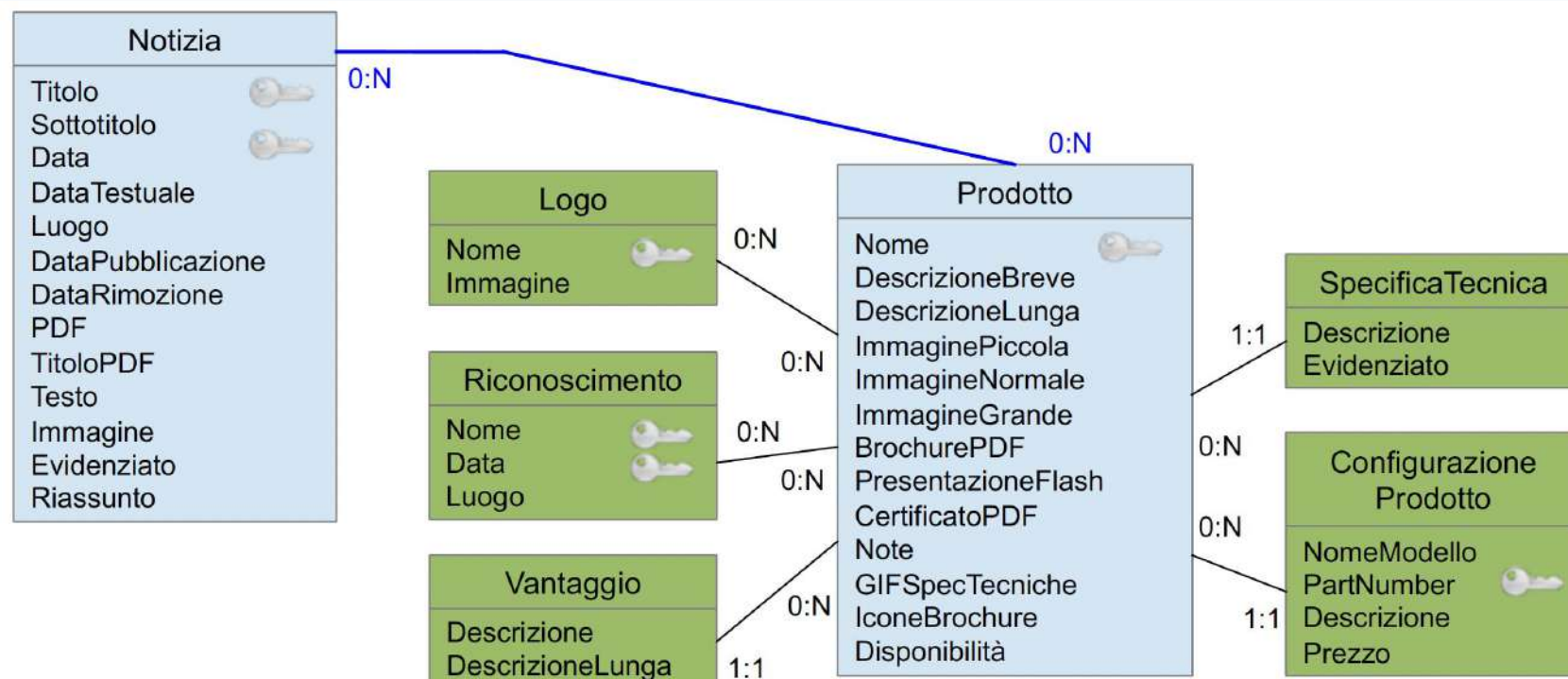


- I sottoschemi di interconnessione prevedono pattern di relazioni, introdotti nello schema dei dati per esprimere associazioni semantiche tra oggetti core
- Derivano direttamente dalle associazioni tra oggetti core dichiarate nel dizionario dei dati
- Ogni associazione semantica nel dizionario dei dati induce una *relazione tra le entità core coinvolte che ha cardinalità molti a molti* (a meno di vincoli specifici dichiarati nel dizionario dei dati)

Sottoschema di «interconnessione» (II)



- Definizione di connessioni semantiche tra sottoschemi «core»

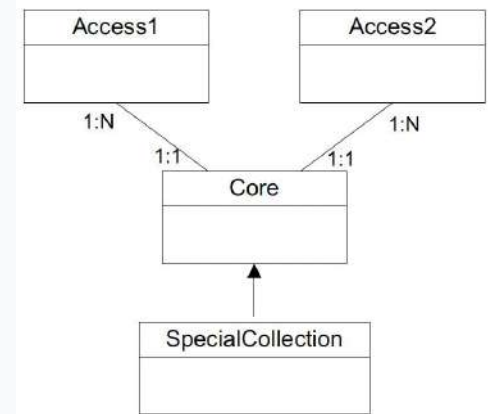




- Le linee guida di buona progettazione «suggerite» dal sottoschema di interconnessione prevedono:
 - La creazione di un link o connessione tra la pagina di dettaglio di un' *entità sorgente* (entità core o componente del sottoschema core) e la pagina di dettaglio dell'entità core interconnessa (*entità destinazione*)
 - Questo implica la definizione di una rotta che, partendo dall'identificatore dell'entità sorgente, estraiga i dettagli dell'entità destinazione



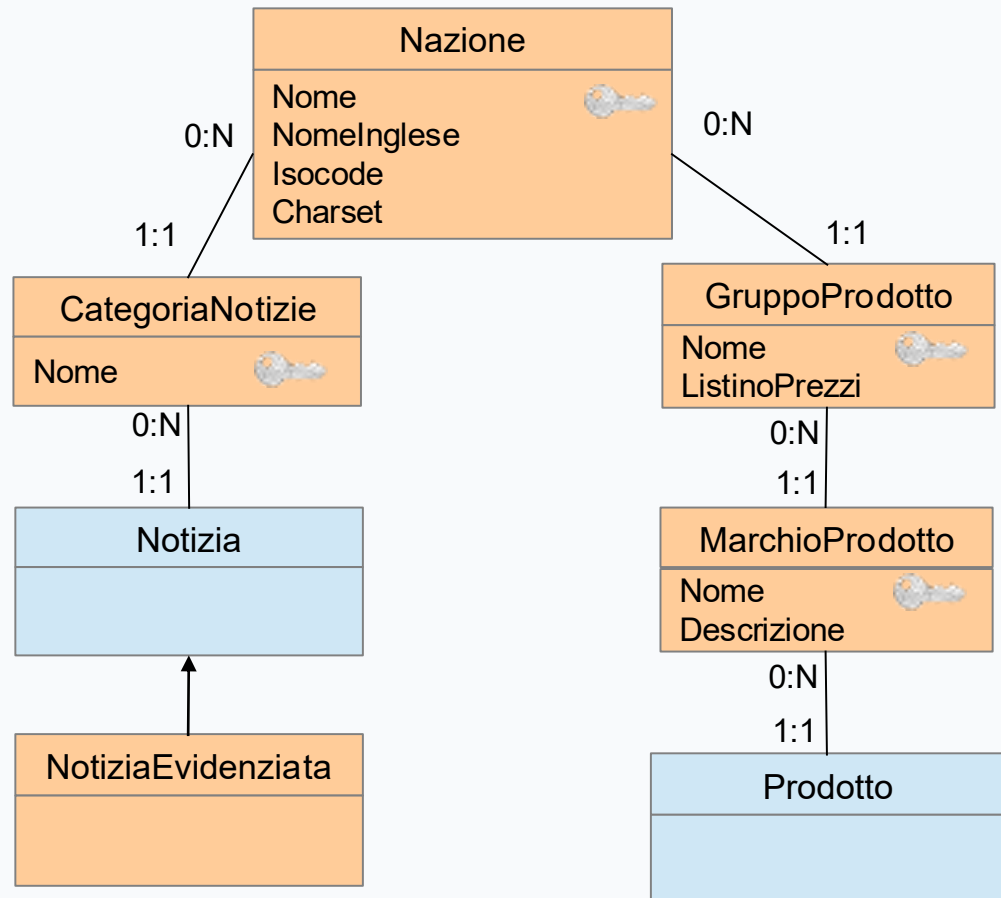
- Oggetti ausiliari, usati per classificare o specializzare gli oggetti core, allo scopo di facilitarne l'accesso in modi diversi:
 - categorizzazioni per costruire gerarchie di indici (categorie-prodotti)
 - meccanismi di ricerca efficienti
 - collezioni di selezioni di oggetti core (le offerte del giorno)
- Uno stesso oggetto core può essere categorizzato o specializzato in diversi modi



Sottoschema di «accesso»



- **Sotto-schema di «accesso»:**
Insieme di entità nello schema E-R, connesse tramite relazioni categorizzazione o gerarchie IS-A sulle entità core

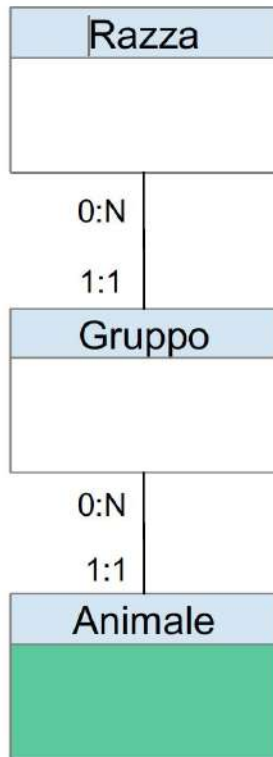




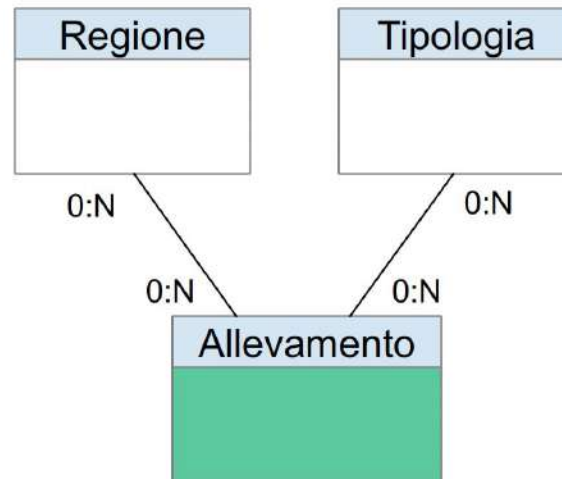
Gli oggetti di categorizzazione possono essere organizzati in tre modi:

- I concetti di categorizzazione possono essere a loro volta categorizzati in modo da formare una ***gerarchia di categorizzazione***
- Lo stesso concetto può essere soggetto a più categorizzazioni originando ***categorizzazioni multiple***
- Lo stesso oggetto di categorizzazione può essere utilizzato per classificare più di un concetto ottenendo una ***categorizzazione condivisa***

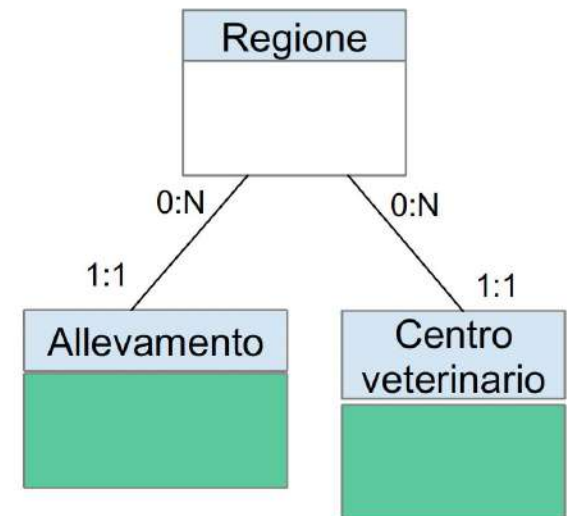
Categorizzazione (II)



gerarchica



multipla



condivisa

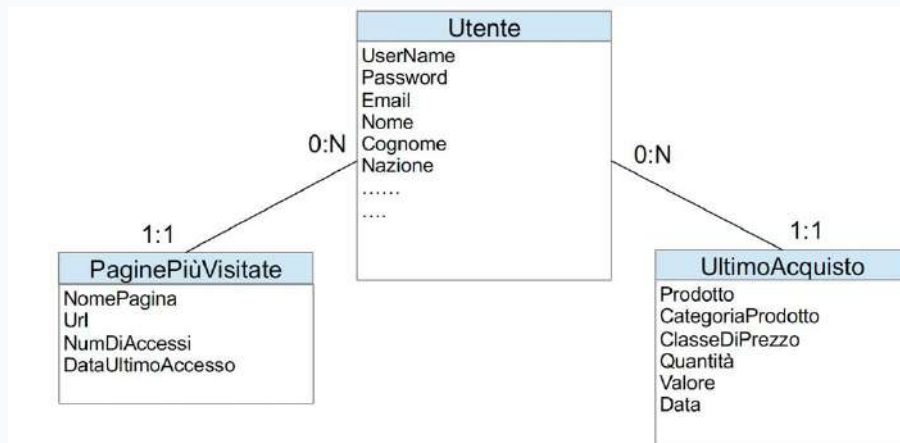


- Le linee guida di buona progettazione «suggerite» dal sottoschema di accesso prevedono:
 - La creazione di un *elemento grafico nel frontend* per utilizzare l'elemento di accesso
 - Un componente progettato per esplorare le categorizzazioni (anche a più livelli)
 - Una form per implementare meccanismi di «*lookup search*» basati sugli attributi delle entità core
 - Ogni elemento prevede una rotta per estrarre le informazioni necessarie per visualizzare l'elemento stesso
 - Un *insieme di rotte* che attivano il meccanismo di ricerca
 - Rotte che restituiscono tutte le istanze di un'entità core assegnata ad una categoria
 - Rotte che sono invocate tramite le «*lookup search form*»

Sottoschemi di «personalizzazione»



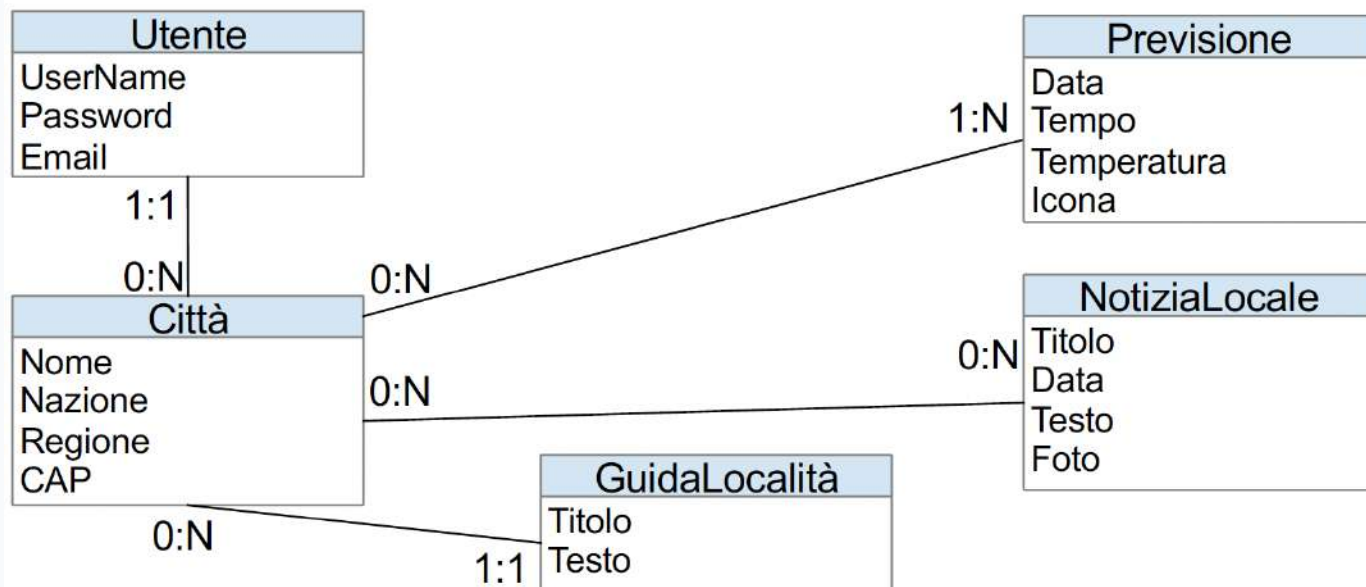
- Un *sottoschema di «personalizzazione»* è costituito da entità e relazioni che descrivono proprietà degli utenti rilevanti per la personalizzazione dell'applicazione:
 - *Dati dei profili utente* (sotto-classi di User nello schema ER)
 - *Relazioni di personalizzazione* con altre entità dello schema ER



Relazioni di «personalizzazione»



- Le **relazioni di personalizzazione** permettono di esprimere a livello dei dati le preferenze degli utenti memorizzate come valori di attributi nell'entità Utente o come istanze di una relazione tra l'entità utente e un'entità di accesso o core





- Le linee guida di buona progettazione «suggerite» dal sottoschema di personalizzazione prevedono:
 - L'inserimento dell'identificatore dell'utente in tutte le rotte che prevedono l'estrazione di istanze dalle entità di accesso e core collegate all'utente dalle relazioni di personalizzazione



- La specifica dei requisiti e la progettazione dei dati comportano lo sviluppo:
 - delle librerie Auth, Security e Users (l'enum UserRole deve includere tutti i gruppi utente individuati)
 - della libreria Database per la connessione al database
 - Di una libreria dedicata al modello dei dati
 - Una **entity** per ogni entità del modello dati
 - Un **controller** per ogni entità del modello ER e uno che includa le eventuali rotte a livello di intero modello (es. i seeder)
 - Un **service**
 - Una serie di **rotte** derivanti dall'identificazione dei casi d'uso e dei sottoschemi dei dati (opportune protette da guardie sui ruoli)
 - Un **DTO** da utilizzare per ogni azione CREATE e UPDATE

Progettazione dell'ipertesto (I)



- Modellazione ad alto livello del frontend di un'applicazione web dinamica e delle interazioni con la logica e i dati di backend
- In WebML, viene utilizzata una notazione visuale semplice, ma formale
- Progettazione di (template di) pagine dinamiche e di interrogazioni per l'accesso e la manipolazione dei dati



- Domande tipiche:
 1. In che modo l'utente deve fruire del contenuto pubblicato tramite il sito?
 2. Quali sono le pagine dell'ipertesto tramite cui l'utente può accedere ai contenuti?
 3. Quale informazione deve essere pubblicata/gestita in ogni pagina?
 4. In che modo i «nodi» dell'ipertesto (per esempio, le pagine o parti di esse) sono collegati tra loro?

Progettazione dell'ipertesto (III)



- **IN**: schema dei dati, mockup dell'interfaccia web, requisiti funzionali e requisiti utente
 - **OUT**: schema formale dell'ipertesto in WebML
-
- Due passi di progettazione:
 1. **Progetto «coarse» (preliminare)** per l'identificazione delle pagine (o gruppi di pagine) e del loro contenuto
 2. **Progetto dettagliato** in cui viene raffinata la progettazione del contenuto informativo delle pagine tramite costrutti specifici



Progettazione di applicazioni web full stack

Progettazione model-driven dei contenuti informativi



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia